Lab 7: Abstraction Date:-07/04/2026

Q1. Write an Abstract Class Vehicle having 4 methods (mix of concrete and abstract methods) to achieve 50 % abstraction. Provide the implementation of abstract class in subclasses, say Car and Truck.

Lab_07 > Q1.java > Q1

```java
package Lab_07;
abstract class Vehicle {

    // Abstract methods (2)
    abstract void start();
    abstract void stop();

    // Concrete methods (2)
    void fuelType() {
        System.out.println(x: "Uses fuel");
    }

    void wheels() {
        System.out.println(x: "Has wheels");
    }
}

class Car extends Vehicle {
    void start() {
        System.out.println(x: "Car starts with key");
    }

    void stop() {
        System.out.println(x: "Car stops with brake");
    }
}

class Truck extends Vehicle {
    void start() {
        System.out.println(x: "Truck starts with button");
    }

    void stop() {
        System.out.println(x: "Truck stops slowly");
    }
}

public class Q1 {
    Run | Debug
    public static void main(String[] args) {
        Vehicle v1 = new Car();
        v1.start();
        v1.stop();

        Vehicle v2 = new Truck();
        v2.start();
        v2.stop();
    }
}
```

OUTPUT:—

```
gauravmathur@Annus-Mac Lab_07 % cd ..
gauravmathur@Annus-Mac LabJava % java Lab_07/Q1
Car starts with key
Car stops with brake
Truck starts with button
Truck stops slowly
```

Q2. Write a program to demonstrate that **abstract classes cannot be instantiated**, but can be referenced using superclass references.

Lab_07 > Q2.java > ...

```java
package Lab_07;
abstract class Animal {
    abstract void sound();
}

class Dog extends Animal {
    void sound() {
        System.out.println(x: "Dog barks");
    }
}

public class Q2 {
    Run | Debug
    public static void main(String[] args) {
        // Animal a = new Animal(); //error ayega isse

        Animal a = new Dog(); //ye allow h
        a.sound();
    }
}
```

OUTPUT:—

```
gauravmathur@Annus-Mac LabJava % javac -d . Lab_07/Q2.java
gauravmathur@Annus-Mac LabJava % java Lab_07/Q2
Dog barks
```

Q3. Write a program to demonstrate that Abstract classes can have **constructors and are called during object creation of subclasses**.

Lab_07 > Q3.java > ...

```java
package Lab_07;

abstract class Person {
    Person() {
        System.out.println("Person constructor called");
    }

    abstract void show();
}

class Student extends Person {
    Student() {
        System.out.println("Student constructor called");
    }

    void show() {
        System.out.println("Student details");
    }
}

public class Q3 {
    Run | Debug
    public static void main(String[] args) {
        Student s = new Student();
        s.show();
    }
}
```

OUTPUT:—

```
gauravmathur@Annus-Mac LabJava % javac -d . Lab_07/Q3.java
gauravmathur@Annus-Mac LabJava % java Lab_07/Q3
Person constructor called
Student constructor called
Student details
```

Q4. Write a program to demonstrate the use of **final methods inside an abstract class** and show that they cannot be overridden.

Lab_07 > Q4.java > Q4

```java
package Lab_07;

abstract class Shape {

    final void display() {
        System.out.println("This is final method");
    }

    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing circle");
    }

    // Cannot override display()
}

public class Q4 {
    Run | Debug
    public static void main(String[] args) {
        Shape s = new Circle();
        s.display();
        s.draw();
    }
}
```

OUTPUT:—

```
gauravmathur@Annus-Mac LabJava % javac -d . Lab_07/Q4.java
gauravmathur@Annus-Mac LabJava % java Lab_07/Q4
This is final method
Drawing circle
```

Q5. Create an interface `Animal` with abstract methods `sound()` and `sleep() and having` **default and static methods**, and implement it in other classes.

Lab_07 > Q5.java > Q5

```java
package Lab_07;

interface Animal {
    void sound();
    void sleep();

    default void eat() {
        System.out.println(x: "Animal eats food");
    }

    static void info() {
        System.out.println(x: "This is Animal interface");
    }
}

class Cat implements Animal {
    public void sound() {
        System.out.println(x: "Cat meows");
    }

    public void sleep() {
        System.out.println(x: "Cat sleeps");
    }
}

public class Q5 {
    Run | Debug
    public static void main(String[] args) {
        Cat c = new Cat();
        c.sound();
        c.sleep();
        c.eat();

        Animal.info();
    }
}
```

OUTPUT:—

```
gauravmathur@Annus-Mac LabJava % javac -d . Lab_07/Q5.java
gauravmathur@Annus-Mac LabJava % java Lab_07/Q5
Cat meows
Cat sleeps
Animal eats food
This is Animal interface
gauravmathur@Annus-Mac LabJava %
```

ava: Ready

Q6. Write a program where an interface is extended by another interface and implemented by a class.

Lab_07 > Q6.java > Q6

```java
package Lab_07;

interface A {
    void methodA();
}

interface B extends A {
    void methodB();
}

class Demo implements B {
    public void methodA() {
        System.out.println(x: "Method A");
    }

    public void methodB() {
        System.out.println(x: "Method B");
    }
}

public class Q6 {
    Run | Debug
    public static void main(String[] args) {
        Demo d = new Demo();
        d.methodA();
        d.methodB();
    }
}
```

OUTPUT:—

```
gauravmathur@Annus-Mac LabJava % javac -d . Lab_07/Q6.java
gauravmathur@Annus-Mac LabJava % java Lab_07/Q6
Method A
Method B
gauravmathur@Annus-Mac LabJava %
```

ava: Ready

Q7. Develop a program where a class implements an interface but does not override all methods and hence must be declared abstract.

Lab_07 > Q7.java > Q7 > main(String[])

```java
package Lab_07;

interface Test {
    void show();
    void display();
}

abstract class Demo implements Test {
    public void show() {
        System.out.println("Show method");
    }
    // display() not implemented--class must be abstract
}

class Demo2 extends Demo {
    public void display() {
        System.out.println("Display method");
    }
}

public class Q7 {
    Run | Debug
    public static void main(String[] args) {
        Demo2 d = new Demo2();
        d.show();
        d.display();
    }
}
```

OUTPUT:—

```
gauravmathur@Annus-Mac LabJava % javac -d . Lab_07/Q7.java
gauravmathur@Annus-Mac LabJava % java Lab_07/Q7
Show method
Display method
```

Q8. Write a Program in which an abstract class providing implementation **(implements)** to an interface, and a concrete class extends the abstract class to provide the implementation to the abstract class.

Lab_07 > Q8.java > ...

```java
package Lab_07;
interface Vehicle {
    void start();
}

abstract class Car implements Vehicle {
    public void start() {
        System.out.println("Car starts");
    }

    abstract void stop();
}

class BMW extends Car {
    void stop() {
        System.out.println("BMW stops");
    }
}

public class Q8 {
    Run | Debug
    public static void main(String[] args) {
        BMW b = new BMW();
        b.start();
        b.stop();
    }
}
```

OUTPUT:—

```
gauravmathur@Annus-Mac LabJava % javac -d . Lab_07/Q8.java
gauravmathur@Annus-Mac LabJava % java Lab_07/Q8
Car starts
BMW stops
gauravmathur@Annus-Mac LabJava %
```